Assignment 6, explained

Fraud imbalance, hypothesis tests, and spam Naive Bayes

A plain-language walkthrough of both problems,
with the numbers from the solution notebook.

SVNIT · Department of Artificial Intelligence
Machine Learning · u24ai028

Two problems, one notebook

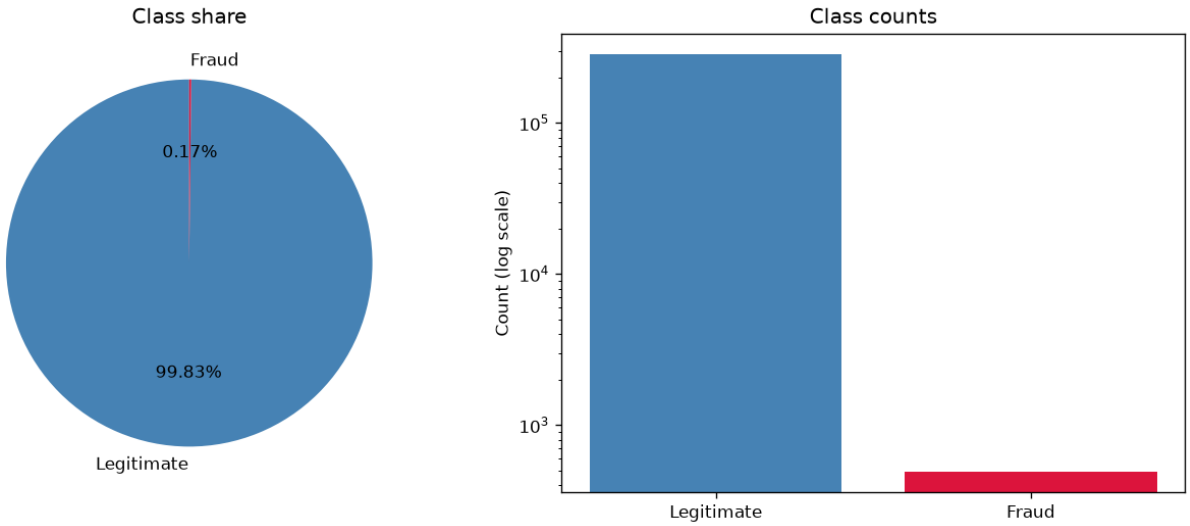
Problem 1 uses 284,807 card payments. 492 of them are fraud, which is 0.173%. The question is how to train a model when one class almost disappears, and how a hypothesis test can tell you which columns actually differ between the classes.

Problem 2 uses 5,572 SMS messages, labelled ham or spam. The text is cleaned, turned into word counts, and then two Naive Bayes models are compared.

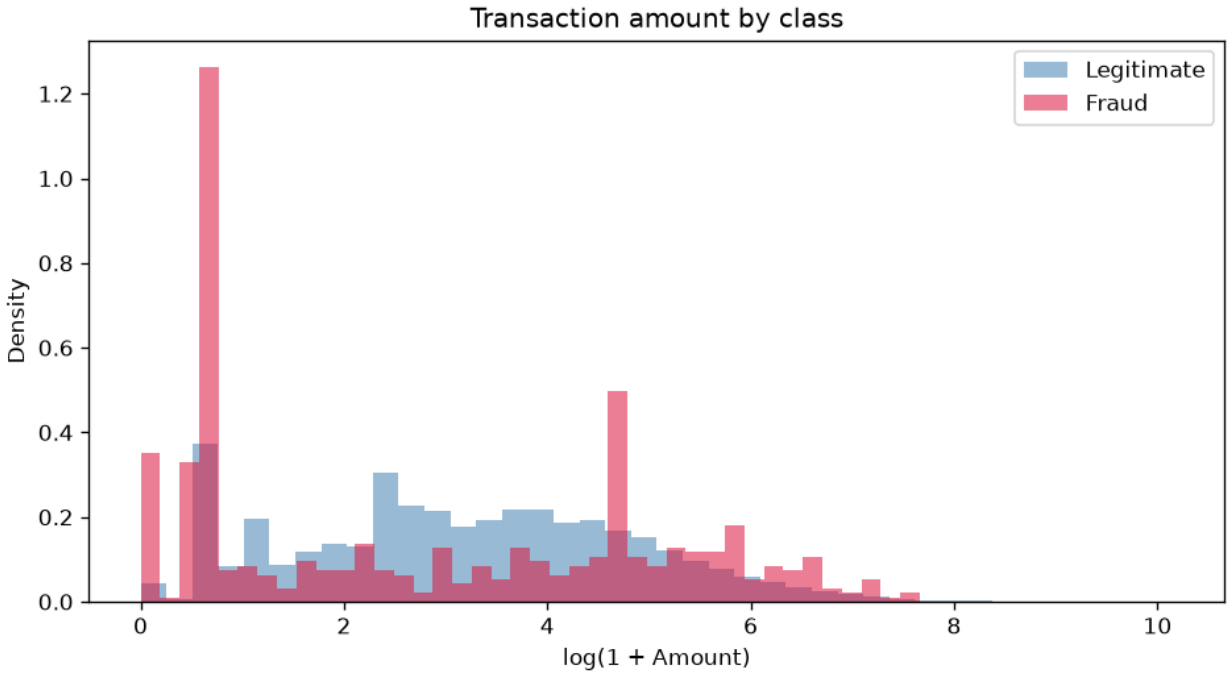
Problem 1A. What is in the file?

Each row is one transaction. Time is seconds since the first payment. V1 to V28 are principal components of the original confidential columns, so they are already numeric and roughly scaled. Amount is the money. Class is 0 for a normal payment and 1 for fraud. There are no missing values.

Fraud amounts are not simply 'the big ones'. Mean fraud amount is $122, mean legitimate amount is $88, but the median fraud is only $9.25. A rule like 'flag large payments' would miss most frauds.



Left: fraud is a sliver, 0.17%. Right: the same counts on a log scale, so the 492 frauds are visible.



Both classes spread across small and large amounts. Amount alone does not separate them.

Problem 1B. Why imbalance breaks accuracy

Predict every payment as legitimate and you are right 284,315 / 284,807 = 99.827% of the time. Recall on fraud is 0%. The loss of a normal model is the sum of mistakes. Almost all of those terms come from legitimate rows, so the easiest way to reduce the loss is to say 'not fraud' everywhere.

The training fix is to change only the training mix. The test set stays at the real 0.17% fraud rate. A resampled test set would be a fake exam.

Stratified 80/20 split, random_state=42: 227,845 train rows (394 frauds) and 56,962 test rows (98 frauds). Time and Amount are on different scales from V1–V28, so StandardScaler is fit on the training columns only.

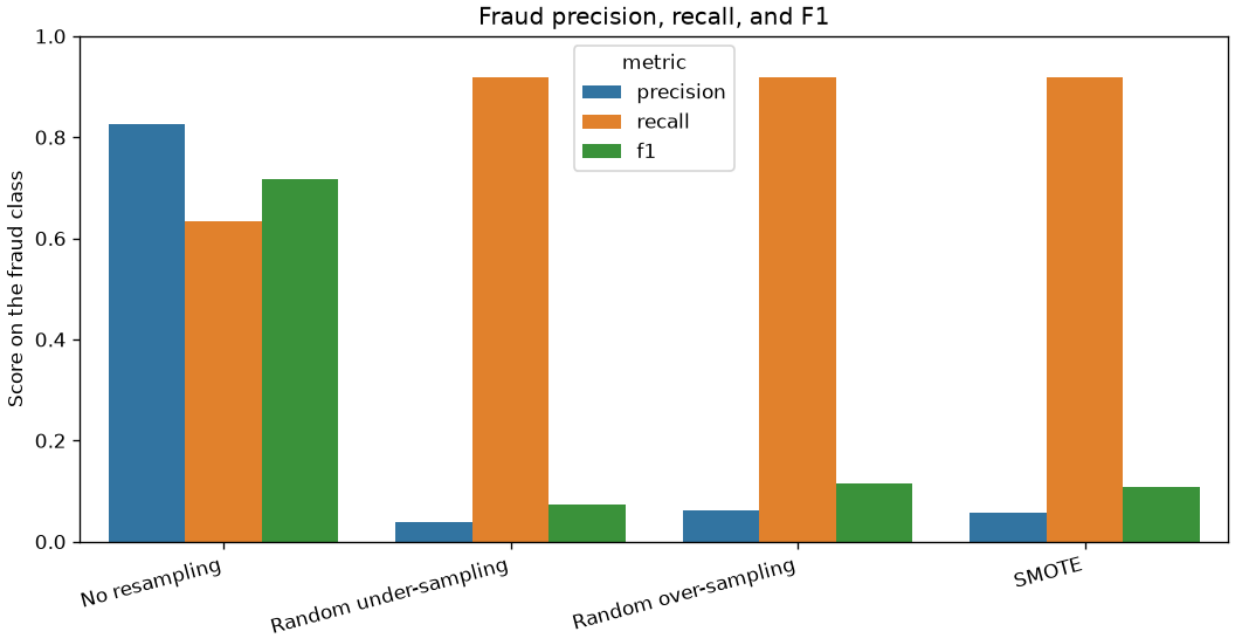
Say this: Accuracy of 99.8% can mean the model caught nothing. For fraud I report precision, recall, and F1 on the fraud class.

Problem 1C. Three ways to rebalance the training set

- Random under-sampling deletes legitimate training rows until 394 frauds and 394 legitimate rows remain (788 rows). Fast, but most normal-payment patterns are thrown away.
- Random over-sampling copies the 394 frauds until there are 227,451 of them. The training set becomes 454,902 rows. Nothing new is invented; the same frauds are repeated.
- SMOTE invents a fraud on the line between a real fraud and one of its nearest fraud neighbours, until the counts match. Same size as over-sampling, 454,902 rows.

The same logistic regression is trained four times. The only change is the training mix. Scores below are on the original test set, fraud class.

Training mix	Precision	Recall	F1
No resampling	0.827	0.633	0.717
Under-sampling	0.038	0.918	0.074
Over-sampling	0.061	0.918	0.115
SMOTE	0.058	0.918	0.109



Sampling lifts recall from 0.63 to 0.92 and knocks precision down near 0.05. F1 follows precision.

Problem 1D. What to conclude

Which method worked best?

By fraud F1, no resampling wins (0.717). It catches 63% of the 98 test frauds and 83% of its alarms are real. All three samplers catch 92% of frauds, but only about 4 to 6 of every 100 alarms are real, so F1 falls below 0.12.

Among the samplers, random over-sampling is slightly ahead of SMOTE, and under-sampling is last, because it also deletes legitimate examples. SMOTE did not beat plain copying: 394 frauds in 30 dimensions is a thin cloud, so a synthetic neighbour is not much new information. Balancing all the way to 227,000 fake frauds also makes the model too willing to shout 'fraud'.

If the bank's real goal is 'miss almost no fraud', a sampler is still useful, but the 0.5 cutoff should then be raised until the false alarms are bearable. F1 at the default cutoff is the wrong only score for that goal.

Which metrics matter?

- Recall: a missed fraud costs money. This is the first number a fraud team asks for.
- Precision: every false alarm is a blocked card and a phone call.
- F1: one number that is bad if either of those is bad.
- Accuracy: ignore it as a headline. 99.83% is the score of doing nothing.

How a hypothesis test helps choose features

Mann-Whitney U asks, for each column, whether fraud values tend to sit higher or lower than legitimate values. It does not assume a bell curve, which matters because Amount is skewed. The test is run on the training set only.

With 200,000 rows, 29 of 30 columns have $p < 0.05$. A tiny shift becomes 'significant' just because the sample is huge. The useful number is the effect size (rank-biserial r, from -1 to 1). Near 0 means the classes are mixed together on that column.

Feature	Effect r	Reading
V14	-0.896	fraud tends to be much lower
V4	+0.884	fraud tends to be much higher
V12	-0.865	fraud tends to be much lower
V10	-0.831	fraud tends to be much lower
V11	+0.825	fraud tends to be much higher

Keep the columns with large |r|. Do not rank columns by p-value. A column with r near 0 can be dropped even if its p-value is tiny.



Red bars: fraud is higher. Blue bars: fraud is lower. Longer bars are the features worth keeping.

Say this: I do not trust accuracy here. Sampling raises recall to 92% and destroys precision. V14 and V4 separate the classes; the p-value does not, because almost every p-value is tiny.

Problem 2. SMS spam

5,572 messages: 4,825 ham and 747 spam. Cleaning is three steps. Lowercase, so Free and free match. Strip punctuation, so free! and free match. Drop English stop words (the, a, to), which show up in both classes.

CountVectorizer builds the vocabulary from the training messages only (6,964 words), then turns each message into a row of numbers. binary=True makes that row 0/1 (word absent or present) for Bernoulli NB. The default keeps the count for Multinomial NB. The split is a stratified 80/20: 4,457 train, 1,115 test. alpha=1 is Laplace smoothing, so a word never seen in spam does not force the whole spam probability to zero.

Model	Accuracy	Precision	Recall	F1
Bernoulli NB	96.59%	0.991	0.752	0.855
Multinomial NB	97.76%	0.949	0.879	0.913

Multinomial NB is better on spam F1 (0.913 vs 0.855). It catches more spam (recall 0.88 vs 0.75). Bernoulli is slightly more precise.

Bernoulli NB treats each word as a yes/no event. Writing 'free' five times is the same evidence as writing it once. That fits a binary bag of words, and it fits short SMS where presence matters more than repetition. Multinomial NB treats the message as a bag of tokens and uses the count. A repeated spam word is repeated evidence. That fits raw counts, or TF-IDF, which is a weighted count. On SMS most counts are already 0 or 1, so the models land close. Multinomial still wins when a spam word is repeated.

Say this: Multinomial NB on word counts wins, F1 0.91. Bernoulli only records whether each word appears, so it misses the extra evidence in a repeated spam word.